
CPSC 436Q - QUANTUM COMPUTING

量子计算

Author

Wenyou (Tobias) Tian

田文友

University of British Columbia

英属哥伦比亚大学

2025

Contents

1	Introduction to Quantum Computing - 量子计算初探	4
2	Randomized Computation - 随机计算	6
2.1	A Simple Example	6
2.2	NAND Tree Problem	6
2.3	Randomized vs. Deterministic Algorithms	7
2.4	Randomized State - 随机态	7
2.5	Randomized Operation - 随机过程	8
2.6	Other Important Concepts	8
3	Basics of Quantum Information Processing - 量子信息处理初步	9
3.1	Complex Numbers - 复数	9
3.2	Quantum State - 量子态	9
3.3	Kronecker Product and Dirac Notation	9
3.4	Quantum Operations - 量子操作	10
3.5	Quantum Circuit - 量子电路	11
3.6	Unitary Matrix in Quantum Computation - 量子计算中的酉矩阵	11
4	CHSH Game	12
4.1	Quantum Strategy - 量子策略	12
4.2	Partial Measurement - 部分测量	14
4.3	Alternative Analysis	15
5	Deutsch's Problem	16
5.1	Quantum Oracle	16
6	Deutsch-Jozsa Problem	18
7	Simon's Problem	20
7.1	Quantum Strategy	20
8	Shor's Algorithm	23
8.1	Modulo Operations and Relevant Mathematical Background	23
8.2	Quantum Fourier Transform	24
9	Grover's Algorithm	26
9.1	Limitations of Quantum Algorithm in Query Models	27
10	Quantum Error Detection and Correction - 量子错误探测与改正	29
10.1	Classical Error Correction	29
10.2	Quantum Error Detection and Correction	29
10.2.1	Bit-flip Error	30
10.2.2	Phase-flip Error	30
10.2.3	Shor's Code	31

10.3 How to Measure Stabilizers	32
---	----

1 Introduction to Quantum Computing - 量子计算初探

In classical computation, *bits* (0 or 1) are used to represent information through whether or not there is current flowing through a transistor. Transistors are classical objects.

In quantum computation, *qubits* are used to represent information. This may take many forms: atomic qubits, like electrons (ground state to be 0 and excited state to be 1); photonic qubits, where the position of a photon on the *dual rail* decides it representing 0 or 1. Other forms include super-conducting circuits or artificial atoms where the quantum property of currents is leveraged. A good qubit is expected to interact properly with the experimenter (e.g. super-conducting circuits), and not interact with the environment (e.g. atoms and photons).

Classical computations have different approaches. The most common one is *deterministic classical computation*, where the output is deterministic for a given input. Consider a piece of 2-bit information, 01, we can encode this using a 2^2 -dimension vector:

$$\begin{pmatrix} 0 & 1 & 0 & 0 \end{pmatrix}^\top$$

where the each element represents a 2-bit state – 00, 01, 10, 11 – respectively. The 1 for 01 shows that this is a *deterministic state*.

Another approach is *randomized computation*, which can be naïvely interpreted as the combination of deterministic computation and the ability to flip coins. Consider we decide a 3-bit state by flipping 2 fair coins for the first two bits, and set the last bit to be 0, then the 3-bit state can be represented by a 2^3 -dimension vector:

$$\begin{pmatrix} \frac{1}{4} & 0 & \frac{1}{4} & 0 & \frac{1}{4} & 0 & \frac{1}{4} & 0 \end{pmatrix}^\top$$

where each element represents a 3-bit state – 000, 001, ..., 111 – respectively, and the number for each element represents the *probability* of the state being in that state.

Notice that an operation like XOR physically operates on n -bits, but abstractly and analytically, it acts on a $2^n \times 1$ -dimension vector. We use this mathematical (linear algebraic) idea extensively.

Contrary to classical computations, quantum computations relies on quantum states, where a state of n qubits is described as a vector of length 2^n consisting *complex numbers*. The origin of complex number and negative numbers in quantum states is the single-particle double-slit experiment that shows interference of a single particle with itself.

We have discussed the general contrast between classical computation and quantum computation, and the different approaches in classical computations. Now we wish to introduce our motivation to develop quantum computation methods.

There are *certain* problems, which quantum computers, using *some* quantum algorithm, can solve faster than classical computers, using *any* classical algorithms (not mathematically proven).

Some common quantum algorithms include:

1. Satisfiability (SAT): best classical algorithm – $\mathcal{O}(2^n)$, best quantum algorithm (Grover's) – $\mathcal{O}(\sqrt{2^n})$. There is a *polynomial speedup*.
2. Factoring problem: best classical algorithm – $2^{\mathcal{O}(n^{\frac{1}{3}})}$, best quantum algorithm (Shor's) – $\mathcal{O}(n^3)$,

where n is the number of input digits in binary. There is a *super-polynomial speedup*.

3. Simulating quantum systems: best classical algorithm – $t \cdot 2^n$, best quantum algorithm – $t \cdot \text{poly}(n)$, where t is the time it takes to evolve from the initial state to the final state. One such system of interest is the covalent bond in a hydrogen molecule.

As of right now, we have scaled up to about 100 *useful* qubits, and about 1000 2-qubit gates. IBM projects in 2029, we can have 100 million gates on 200 *logical* qubits (each logical qubit consists of about 100 to 1000 normal qubits). In very recent developments, Google announced its 105-qubit Willow quantum processor with a 2-qubit gate error rate of 0.33% (meaning they have about $\frac{1}{0.33\%}$ gates). For comparison purposes, the benchmark for factoring RSA-2048 requires about 20 million qubits and 3 billion gates; the benchmark for simulating a compound like FeMoco requires about 4 million qubits and 5 billion operations.

2 Randomized Computation - 随机计算

To demonstrate how randomized computation speeds up compared to deterministic computation, we can consider the following examples.

2.1 A Simple Example

Consider that you are given a string of n bits, which are promised to be either all 0s or exactly half of the bits are 1s, find an algorithm to decide which case you are in.

Deterministically, the time complexity is $\Theta(n)$ as you need to try at least $\frac{n}{2}$ times, and at most n times. With a randomized algorithm, the time complexity is $\mathcal{O}(1)$, where we can obtain the correct answer with high probability. The description is simple,

While we do not see a 1, we select a bit randomly. If this bit is 1, we say we are in the second case. Otherwise, we say we are in the first case, where the string is all 0s.

We run the above algorithm a constant number of times, say c . If the string is indeed all 0s, the algorithm above is always correct. If the string is half 1s, then the algorithm is correct with a probability of

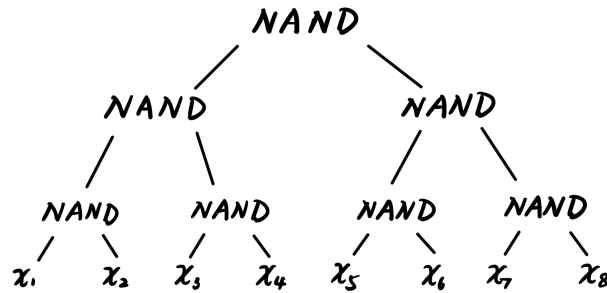
$$\begin{aligned}\Pr(\text{success}) &= \Pr(\text{seeing 1 once among } c \text{ times}) \\ &= 1 - \Pr(\text{seeing all 0}) \\ &= 1 - \left(\frac{1}{2}\right)^c\end{aligned}$$

When the string is half 1s, we can obtain the correct output with arbitrarily high probability as we increase c .

This is a type of *Monte Carlo* algorithm, where we have a fixed runtime, but can be wrong.

2.2 NAND Tree Problem

Consider an input of n bits where $n := 2^h$. Say $h = 3$, we consider a balanced binary tree



We want to know the output of the root node.

Deterministically, the time complexity is $\Theta(n) = \Theta(2^h)$.

As for the randomized algorithm, we first consider a step of the algorithm, denoted as $\mathcal{A}_h$, where h represents the height of the tree. This single step does one thing: starting from the root, examine left or right branch uniformly at random; if sees 0, we output 1; if sees 1, we examine the other branch and

the output NAND of both examinations. This is a recursive randomized algorithm. We further denote $\alpha_b(h)$ to be the expected complexity of $\mathcal{A}_h$ if input $\mathbf{x}$ maps to b . Then, we have:

$$\begin{aligned}\alpha_0(h) &\leq 2\alpha_1(h-1) \\ \alpha_1(h) &\leq \max \begin{cases} \alpha_0(h-1) \\ \alpha_0(h-1) + \frac{1}{2}\alpha_1(h-1) \\ \alpha_0(h-1) + \frac{1}{2}\alpha_1(h-1) \end{cases} \\ &\leq 2\alpha_1(h-2) + \frac{1}{2}\alpha_1(h-1)\end{aligned}$$

These inequalities stand from the NAND operations where:

$$\begin{aligned}0 \text{ NAND } 0 &= 1 \\ 0 \text{ NAND } 1 &= 1 \\ 1 \text{ NAND } 0 &= 1 \\ 1 \text{ NAND } 1 &= 0\end{aligned}$$

And this algorithm solves to have a complexity of $\sim \mathcal{O}(1.686^h)$.

This is a type of *Las Vegas* algorithm, where we have unfixed runtime, but the output is always correct.

2.3 Randomized vs. Deterministic Algorithms

A randomized algorithm can be thought of as choosing from some distribution of deterministic algorithms. If we require the randomized algorithms to have both fixed runtime and the output to be always correct, then there is no advantage from randomization.

We cover randomized algorithms because many quantum algorithms employ randomness and we want to show that quantum algorithms can be strictly faster than randomized algorithms (otherwise, speedup from randomization would be enough).

2.4 Randomized State - 随机态

Definition 2.1. A *randomized state* on n bits is described by a column vector of length 2^n

$$\vec{p} = (p_{00\dots 0}, p_{00\dots 1}, \dots, p_{11\dots 1})^\top$$

such that it satisfies two conditions:

1. Normalization: $\sum_{\mathbf{x}} p_{\mathbf{x}} = 1$.
2. Non-negativity: $\forall p_{\mathbf{x}}, p_{\mathbf{x}} \geq 0$

If exactly one of $p_{\mathbf{x}} \neq 0$, we refer to $\vec{p}$ as a *deterministic state*, corresponding to bitstring $\mathbf{x}$.

Note that *doing operations on randomized states in different orders can change the out-*

come. We can also consider doing randomized operations on randomized states as well.

2.5 Randomized Operation - 随机过程

Definition 2.2. A *randomized operation* (a.k.a stochastic operation/matrix) acting on n bits is described by a $2^n \times 2^n$ matrix A , such that:

1. All entries of A are non-negative,
2. Sum of every column is 1.

This implies that every column is a randomized state. If every column is a deterministic state, then we say A is a *deterministic operation*.

For example, a coin flip on 1 bit can be characterized as

$$A = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{pmatrix}$$

Lemma 2.1. A randomized operation that acts on a randomized state will yield a randomized state.

2.6 Other Important Concepts

The *tensor product* of randomized states can be regarded as the following. Consider a 2-bit state, where the first bit is determined by a fair coin, while the second bit is set to 0. If we wish to describe the final state, we have:

$$\begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ \frac{1}{2} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ 0 \\ 0 \\ \frac{1}{2} \end{pmatrix}$$

Similarly, an example of the tensor product of randomized operations is the following:

$$\text{NOT} \otimes \mathbb{I}$$

A *randomized circuit* is a circuit consisting of randomized operations.

Measuring a randomized state on n bits, $\vec{p}$, is a procedure that outputs $\mathbf{x} \in \{0, 1\}^n$ with probability $p_{\mathbf{x}}$.

The process of randomized computation consists of input bits and *ancillas* that goes through a randomized circuit and obtain an output through a measurement.

For randomized computation, the Toffoli gate and the CNOT operation is universal – any randomized computation can be decomposed into a sequence of these 2 operations to arbitrary precision.

3 Basics of Quantum Information Processing - 量子信息处理初步

3.1 Complex Numbers - 复数

Let $a, b, c, d, r, \theta \in \mathbb{R}$, we introduce the imaginary unit i where $i^2 = -1$.

A complex number is in the form of $z = a + bi$ with the following arithmetic properties:

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i$$

$$|a + bi| = \sqrt{a^2 + b^2}$$

$$e^{i\theta} = \cos \theta + i \sin \theta$$

$$(a + bi)^* = a - bi$$

Let $z, z_1, z_2 \in \mathbb{C}$, then we have:

$$|z|^2 = z \cdot z^*$$

$$z_1 = r_1 e^{i\theta_1}$$

$$z_2 = r_2 e^{i\theta_2}$$

$$z_1 z_2 = (r_1 r_2) e^{i(\theta_1 + \theta_2)}$$

3.2 Quantum State - 量子态

Definition 3.1. A quantum state of n qubits is described by a column vector of length 2^n ,

$$\vec{\alpha} = (\alpha_{00\dots 0}, \alpha_{00\dots 1}, \dots, \alpha_{11\dots 1})^\top$$

such that $\alpha_{\mathbf{x}} \in \mathbb{C}$, and $\sum_{\mathbf{x}} |\alpha_{\mathbf{x}}|^2 = 1$.

It is easy to see any deterministic state is a quantum state.

3.3 Kronecker Product and Dirac Notation

Definition 3.2. Let $A \in \mathbb{C}^{m_1 \times m_2}$ and $B \in \mathbb{C}^{n_1 \times n_2}$, then

$$A \otimes B = \begin{pmatrix} a_{11}B & a_{12}B & \cdots & a_{1,m_2}B \\ a_{21}B & a_{22}B & \cdots & a_{2,m_2}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m_1 1}B & a_{m_1 2}B & \cdots & a_{m_1, m_2}B \end{pmatrix} \in \mathbb{C}^{(m_1 \times n_1)(m_2 \times n_2)}$$

It is conventional to write $\vec{\alpha}$ as $|\alpha\rangle$:

$$|\alpha\rangle = \sum_{\mathbf{x}} \alpha_{\mathbf{x}} |x_1 x_2 \cdots x_n\rangle$$

In $\mathbb{C}^d$ for $d > 2$, it is conventional to write:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, |1\rangle = \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix}, |d-1\rangle = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}$$

The properties of the Kronecker product consists of the following:

Let $A, B, C, D \in \mathbb{C}^{m \times n}$, and let $\alpha, \beta \in \mathbb{C}$.

1. Bilinearity: if A and B have same dimensions,

$$C \otimes (\alpha A + \beta B) = \alpha C \otimes A + \beta C \otimes B$$

2. Associativity

$$(A \otimes B) \otimes C = A \otimes (B \otimes C)$$

3. Mixed product: if A, C have same dimensions, and B, D have same dimensions

$$(A \otimes B)(C \otimes D) = AC \otimes BD$$

With these setup, we can see that if $A \in \mathbb{C}^{d \times d}$, then

$$A|i\rangle = \sum_{j=1}^d A_{ji} |j\rangle$$

for all $i \in \{1, \dots, d\}$.

3.4 Quantum Operations - 量子操作

Definition 3.3. A square matrix $\mathcal{U}$ is unitary if $\mathcal{U}^\dagger \mathcal{U} = \mathbb{I}$.

Lemma 3.1. $|u_1\rangle, |u_2\rangle, \dots, |u_d\rangle \in \mathbb{C}^d$ form an orthonormal basis if there exists a unitary matrix $\mathcal{U}$ such that $|u_i\rangle = \mathcal{U}|i\rangle$

Definition 3.4. A quantum operation on d -dimensional quantum states is described by a $d \times d$ unitary matrix.

Common elementary gates include

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{pmatrix}, \text{CNOT, Toffoli}$$

where H is the **Hadamard** gate, and X, Y, Z are **Pauli gates**.

3.5 Quantum Circuit - 量子电路

Definition 3.5. A quantum circuit is a sequence of quantum operations.

Definition 3.6. Measuring a quantum state of n -qubits *in the quantum basis* $|\psi\rangle = \sum_{\mathbf{x}} \alpha_{\mathbf{x}} |\mathbf{x}\rangle$ refers to a process that returns an n -bit classical string $\mathbf{x} \in \{0, 1\}^n$ with probability $|\alpha_{\mathbf{x}}|^2$.

Common universal gate sets for a quantum circuit include

$$\{H, \text{CNOT}, T\}, \{H, \text{Toffoli}\}$$

3.6 Unitary Matrix in Quantum Computation - 量子计算中的酉矩阵

We refer the *bra* notation to be row vectors (e.g. $\langle\psi|$) and *ket* notation to be column vectors (e.g. $|\psi\rangle$). Let $|\psi\rangle, |\phi\rangle \in \mathbb{C}^d$, we have $\langle\psi| = |\psi\rangle^\dagger$, $\langle\phi|\psi\rangle = \langle\psi| \cdot |\psi\rangle$. The norm of $|\psi\rangle$ is $\| |\psi\rangle \| = \sqrt{\langle\psi|\psi\rangle}$.

Lemma 3.2. For operations $\text{op} \in \{*, \top, \dagger\}$, we have

$$(A + B)^{\text{op}} = A^{\text{op}} + B^{\text{op}}$$

$$(A \otimes B)^{\text{op}} = A^{\text{op}} \otimes B^{\text{op}}$$

and $(AB)^* = A^* B^*$, $(AB)^\top = B^\top A^\top$, $(AB)^\dagger = B^\dagger A^\dagger$

Lemma 3.3. Suppose $|\psi\rangle = \sum_{i=1}^d \alpha_i |i\rangle \in \mathbb{C}^d$, then $|\psi\rangle$ is a quantum state if and only if $\langle\psi|\psi\rangle = 1$.

Theorem 3.1. Let $|u_1\rangle, |u_2\rangle, \dots, |u_d\rangle \in \mathbb{C}^d$, then $\langle u_i | u_j \rangle = \delta_{ij}$ for all $i, j \in \{1, \dots, d\}$ if and only if they form an orthonormal basis.

Equivalently, a $d \times d$ matrix is unitary if and only if the columns of A are unit vectors and are pair-wise orthogonal.

Theorem 3.2. A $d \times d$ complex matrix A maps a d -dimensional quantum state to another d -dimensional quantum state if and only if A is unitary.

4 CHSH Game

The CHSH game, also demonstrating the violation of Bell inequality, has the following set up.

We have 3 people, Alice, Bob and the referee. The referee will distribute x, y to Alice and Bob respectively such that x, y are drawn uniformly at random from the set $\{0, 1\}$. Without discussing, Alice and Bob must give $a, b \in \{0, 1\}$ to the referee respectively in their own choices. Alice and Bob wins if

$$a \oplus b = x \wedge y$$

One naïve strategy that Alice and Bob can use is always choosing $a = b = 0$, this way they have a $\frac{3}{4}$ chance of winning the game.

If we think about it more generally, and discuss a deterministic strategy, that is:

$$\begin{aligned} a : \{0, 1\} &\rightarrow \{0, 1\} \\ x &\mapsto 0 \text{ or } 1 \\ b : \{0, 1\} &\rightarrow \{0, 1\} \\ y &\mapsto 0 \text{ or } 1 \end{aligned}$$

We can demonstrate that for all possible pairs of (x, y) , it is impossible to satisfy $a(x) \oplus b(y) = x \wedge y$ at the same time. There is at most a $\frac{3}{4}$ chance of winning the game.

What if we employ a randomized strategy? Before Alice and Bob go into the game, they sample some random variable $\lambda \in \mathbb{R}$ with probability density p_λ , and choose a deterministic strategy a_λ and b_λ . If we draw out each λ_i for all $i \in \{1, \dots, n\}$, each path is a deterministic strategy with winning probability at most $\frac{3}{4}$. Thus, the probability of winning the game using randomized strategy:

$$\Pr(\text{winning}) \leq \sum_i p_{\lambda_i} \times 0.75 = 0.75$$

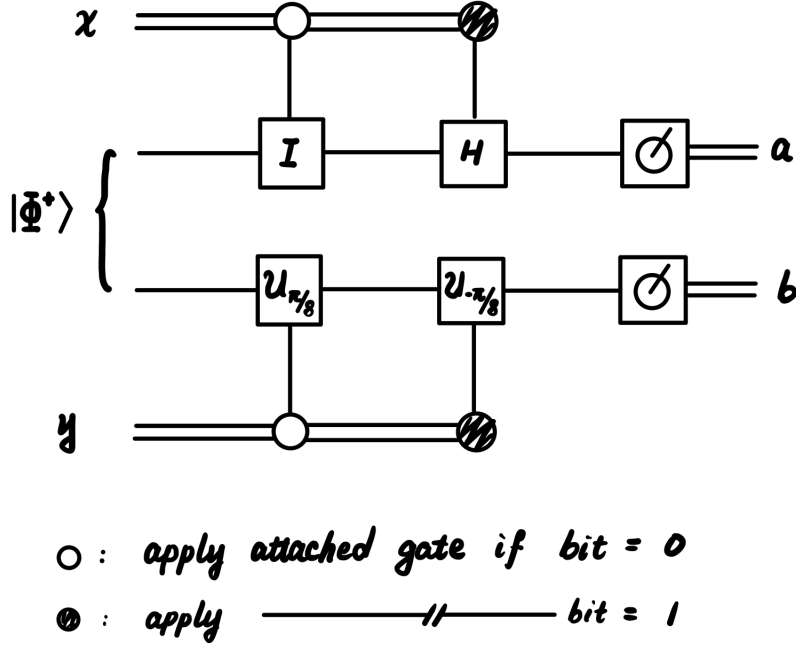
Thus, any *randomized strategy* with *no communication* wins with a probability at most 0.75; any *hidden variable* theory that is also *local* wins with a probability at most 0.75.

4.1 Quantum Strategy - 量子策略

Theorem 4.1. By employing a quantum strategy, Alice and Bob can win the game with probability $\cos^2(\frac{\pi}{8})$.

Definition 4.1. A 2-qubit state $|\psi\rangle$ is *entangled* if $|\psi\rangle$ cannot be written as $|\psi_1\rangle \otimes |\psi_2\rangle$ where $|\psi_i\rangle$ are 1-qubit states.

The quantum strategy is described by a quantum circuit:



where $\mathcal{U}_\theta$ is the following (it can be interpreted as a rotation matrix by angle θ *clockwise*):

$$\mathcal{U}_\theta = \begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix}$$

and $|\Phi^+\rangle$ is also called the EPR pair specified to be:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

We provide one example case to demonstrate the above probability, where $x = 1$ and $y = 0$:

$$\begin{aligned} |\Phi^+\rangle &\rightarrow (H \otimes \mathcal{U}_{\frac{\pi}{8}}) |\Phi^+\rangle \\ &= \mathbb{I}(H \otimes \mathcal{U}_{\frac{\pi}{8}}) |\Phi^+\rangle \\ &= \sum_{\mathbf{x}} |\mathbf{x}\rangle \langle \mathbf{x}| H \otimes \mathcal{U}_{\frac{\pi}{8}} |\Phi^+\rangle \\ &= \sum_{\mathbf{x}} \langle \mathbf{x}| H \otimes \mathcal{U}_{\frac{\pi}{8}} |\Phi^+\rangle |\mathbf{x}\rangle \end{aligned}$$

Since $x \wedge y = 0$, we want $a \oplus b = 0$, meaning that either $a = b = 0$ or $a = b = 1$, that is, we only care about the coefficients for $\mathbf{x} = 00, 11$, which is:

$$\Pr(\mathbf{x} = 00, \mathbf{x} = 11) = |\langle 00| H \otimes \mathcal{U}_{\frac{\pi}{8}} |\Phi^+\rangle|^2 + |\langle 11| H \otimes \mathcal{U}_{\frac{\pi}{8}} |\Phi^+\rangle|^2$$

If we expand the first part we have:

$$\begin{aligned}
\langle 00 | H \otimes \mathcal{U}_{\frac{\pi}{8}} | \Phi^+ \rangle &= \frac{1}{\sqrt{2}} \langle 00 | (H \otimes \mathcal{U}_{\frac{\pi}{8}} | 00 \rangle + H \otimes \mathcal{U}_{\frac{\pi}{8}} | 11 \rangle) \\
&= \frac{1}{\sqrt{2}} \langle 00 | ((\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle))(\cos \theta |0\rangle - \sin \theta |1\rangle) \\
&\quad + (\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle))(\sin \theta |0\rangle + \cos \theta |1\rangle)) \\
&= \frac{1}{2}(\cos \theta + \sin \theta)
\end{aligned}$$

If we do the same thing for $\langle 11 | H \otimes \mathcal{U}_{\frac{\pi}{8}} | \Phi^+ \rangle$, we would obtain $\frac{1}{2}(-\sin \theta - \cos \theta)$. We take the square of each value, add them together, and substitute $\theta = \frac{\pi}{8}$, we get the probability of $\cos^2(\frac{\pi}{8})$.

4.2 Partial Measurement - 部分測量

Definition 4.2. Let $m \leq n$ be positive integers. A *partial measurement* of the first m qubits of an n -qubit state $|\psi\rangle = \sum_{\mathbf{x}} \alpha_{\mathbf{x}} |\mathbf{x}\rangle$ is a process that:

1. returns $\mathbf{y} \in \{0, 1\}^m$ with probability

$$p_{\mathbf{y}} := \|(\langle \mathbf{y} | \otimes \mathbb{I}^{n-m}) |\psi\rangle\|^2 = \sum_{\mathbf{x}[1:m]=\mathbf{y}} |\alpha_{\mathbf{x}}|^2$$

2. given $\mathbf{y}$ is returned, the state of the remaining $n - m$ qubits become

$$|\psi_{\mathbf{y}}\rangle := \frac{\langle \mathbf{y} | \mathbb{I}^{n-m} | \psi \rangle}{\sqrt{p_{\mathbf{y}}}}$$

An example of this is given $|\psi\rangle = \sqrt{\frac{1}{2}}|00\rangle + \sqrt{\frac{1}{4}}|01\rangle + \sqrt{\frac{1}{6}}|10\rangle + \sqrt{\frac{1}{12}}|11\rangle$, if we measure the 1st qubit, the probability of getting 0 is $\frac{3}{4}$ and getting 1 to be $\frac{1}{4}$, which both coincidentally have the same leftover state after partial measurement to be:

$$\sqrt{\frac{2}{3}}|0\rangle + \sqrt{\frac{1}{3}}|1\rangle$$

Theorem 4.2. Let $|\phi\rangle$ be any n -qubit state, consider 2 experiments: 1. directly measure the first m qubits, obtaining outcome b and post-measurement state $|\phi_b\rangle$.

2. first apply some $(n - m)$ -qubit unitary $\mathcal{U}$ on the last $n - m$ qubits of $|\phi\rangle$, then measure the first m qubits of $|\phi\rangle$, obtaining outcome c and post-measurement state $|\phi'_c\rangle$.

Then, the distribution of b, c are identical, and $|\phi'_c\rangle = \mathcal{U} |\phi_c\rangle$ for all $c \in \{0, 1\}^m$.

4.3 Alternative Analysis

Given the partial measurement definition, we can re-analyze the CHSH game.

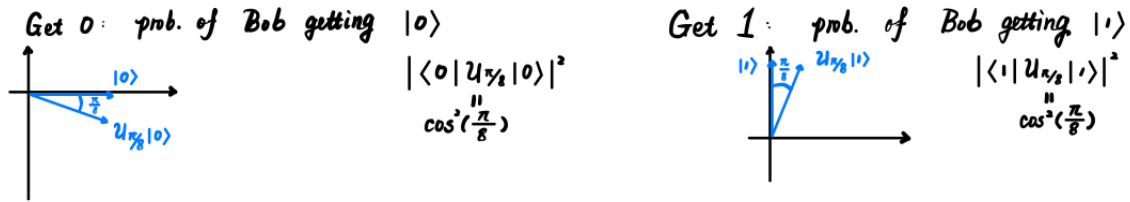
From linear algebra, for two real vectors $|u\rangle$ and $|v\rangle$ of the same dimension, we have:

$$\langle u|v\rangle = \| |u\rangle \| \cdot \| |v\rangle \| \cdot \cos \theta$$

and if they are quantum states, we further have $|\langle u|v\rangle|^2 = \cos^2 \theta$.

We can consider that Alice measuring first, which is a partial measurement on the first qubit, and get a . For each case of (x, y) , we can then consider the probability of Bob measuring the corresponding b so that they can win to be plotted on a 2D Cartesian plane.

An example of this is when $(x, y) = (0, 0)$, Alice has $\frac{1}{2}$ of getting 0 and 1 respectively. And we want to find the probability of Bob measuring 0 and 1 respectively as well, then we have:



This process can be applied to each (x, y) .

5 Deutsch's Problem

We wish to explore quantum algorithms and speedups. The first historic example of quantum speedup is from Deutsch's problem. The setup of this problem is as follows.

Given $f : \{0, 1\} \rightarrow \{0, 1\}$, we ask if f is balanced ($f(0) \neq f(1)$) or constant ($f(0) = f(1)$).

From here, we give some what we mean by comparing the complexity between classical algorithms and quantum algorithms.

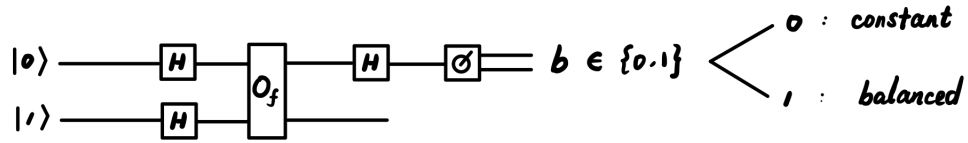
For a classical algorithm, we assume f can only be accessed through queries (which is an evaluation of f at a single point), and each query is unit cost. The complexity of the algorithm is the number of queries needed to solve the problem. To solve the above problem with certainty, 2 queries is needed.

For a quantum algorithm, assume we have the ability to apply a 2-qubit unitary at unit cost, defined by

$$O_f : |\mathbf{x}\rangle |b\rangle \mapsto |\mathbf{x}\rangle |b \oplus f(x)\rangle$$

for all $\mathbf{x}, b \in \{0, 1\}$. Each application of O_f is a quantum query to f . To solve the above problem with certainty, only 1 quantum query is needed.

The quantum circuit is set up as the following:



During this analysis, two mathematical relations is useful:

$$|0 \oplus x\rangle = |x\rangle$$

$$|x\rangle - |1 \oplus x\rangle = (-1)^x(|0\rangle - |1\rangle)$$

The above circuit, with a single quantum query, can solve Deutsch's problem, the mathematical derivation is not included. We may also think of this from a partial measurement perspective, where applying a unitary on the non-measured qubit does not change the probability distribution of the measured qubit, so if we apply another Hadamard gate after the quantum query on the second qubit, we can simplify the state before measurement to be:

$$|\psi'\rangle = (-1)^{f(0)} \begin{cases} |0\rangle & \text{constant} \\ |1\rangle & \text{balanced} \end{cases} \otimes |1\rangle$$

5.1 Quantum Oracle

Given f as a classical circuit, we can systematically and efficiently convert a classical circuit diagram into its corresponding quantum oracle.

Definition 5.1. Let $f : \{0, 1\}^m \rightarrow \{0, 1\}^n$, the quantum oracle of f is the unitary matrix $O_f \in \mathbb{C}^{2^{m+n} \times 2^{m+n}}$ defined by

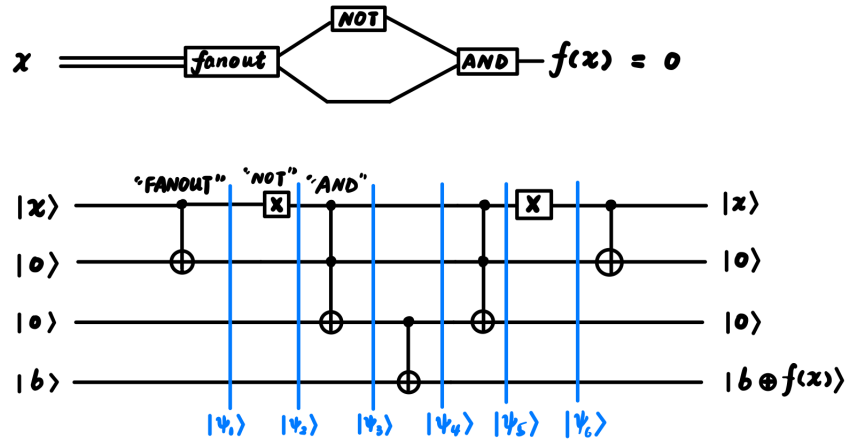
$$O_f : |\mathbf{x}\rangle |\mathbf{y}\rangle \mapsto |\mathbf{x}\rangle |\mathbf{y} \oplus f(\mathbf{x})\rangle$$

Theorem 5.1. Any f can be implemented as a classical circuit involving FANOUT, NOT, AND, OR gates.

Theorem 5.2. Suppose we have the description of classical circuit implementing f in terms of the above gates, we can systematically and efficiently implement O_f using CNOT, X, and Toffoli gates.

More precisely, the implementation takes size that is linear (roughly $2\times$ due to symmetry) in the size (number of gates) of the classical circuit.

Specifically, we map FANOUT to CNOT gate, NOT to X gate, AND to Toffoli gate, and OR can be implemented using Toffoli and X gates (because OR can be implemented using NAND gates). An example is the following:

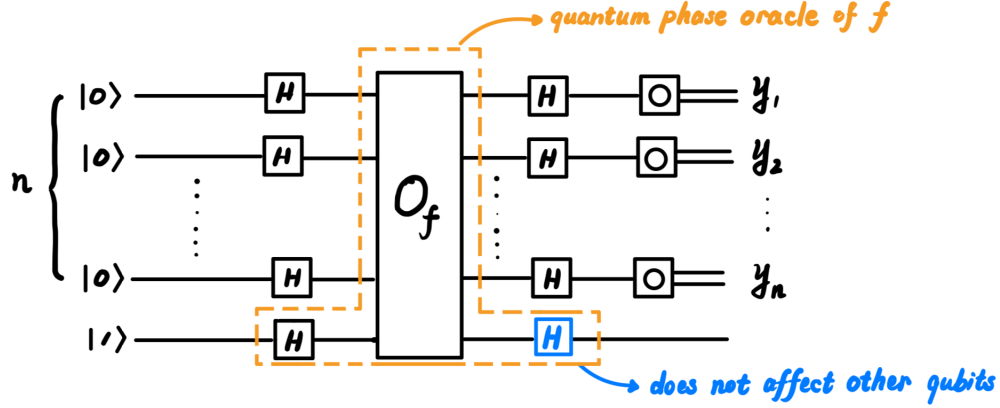


We can consider CNOT “copying” a bit (because $0 \oplus x = x$), X negates a bit (because X maps $|0\rangle$ to $|1\rangle$ and vice versa), Toffoli taking two bits and put the AND result into the third bit (because Toffoli only flips the third bit if both first bit and second bit are 1).

6 Deutsch-Jozsa Problem

Deutsch-Jozsa is an “upgraded” version of Deutsch’s problem. The problem set up is the following. Given $f : \{0, 1\}^n \rightarrow \{0, 1\}$, we are *promised* that it is either constant ($\forall \mathbf{x} \in \{0, 1\}^n, f(\mathbf{x}) = b$) or balanced (half of $\mathbf{x} \in \{0, 1\}^n, f(\mathbf{x}) = 0$, the other half gives 1). We want to decide which case we are in.

The quantum circuit that solves this problem is the following:



$$\text{Output} \begin{cases} \text{"constant"} & \text{if } y_1 = \dots = y_n = 0 \\ \text{"balanced"} & \text{if } y_1, \dots, y_n \neq 0 \end{cases}$$

The *quantum phase oracle* of f mentioned in this circuit is given by:

$$\mathcal{U}_f : \mathbf{x} |b\rangle \mapsto (-1)^{b \cdot f(\mathbf{x})} |\mathbf{x}\rangle |b\rangle$$

One useful lemma for analysis is that

Lemma 6.1. Given $b \in \{0, 1\}$, we have

$$H |b\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^b |1\rangle)$$

This extends to

$$H^n |\mathbf{x}\rangle = \frac{1}{\sqrt{2^n}} \sum_{\mathbf{y} \in \{0,1\}^n} (-1)^{\mathbf{x} \cdot \mathbf{y}} |\mathbf{y}\rangle$$

where

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^n x_i \cdot y_i$$

Another useful mathematical trick is that

$$(-1)^{1 \oplus b} = -(-1)^b$$

where $b \in \{0, 1\}$.

The speedup of this quantum circuit is obvious. If we use a classical query model, we need $2^{n-1} + 1$ times to be certain. In this model, we need one quantum query to be certain.

However, this *does not* mean we have an unconditional provable exponential quantum speedup, because once we instantiate f as a classical circuit, there may be more efficient ways of solving the D-J problem classically, by analyzing the circuit and just querying it.

Being only able to query the circuit for f is the essential simplifying assumption of the query model.

7 Simon's Problem

Simon's problem gives an exponential separation between classical and quantum without demanding certain correctness.

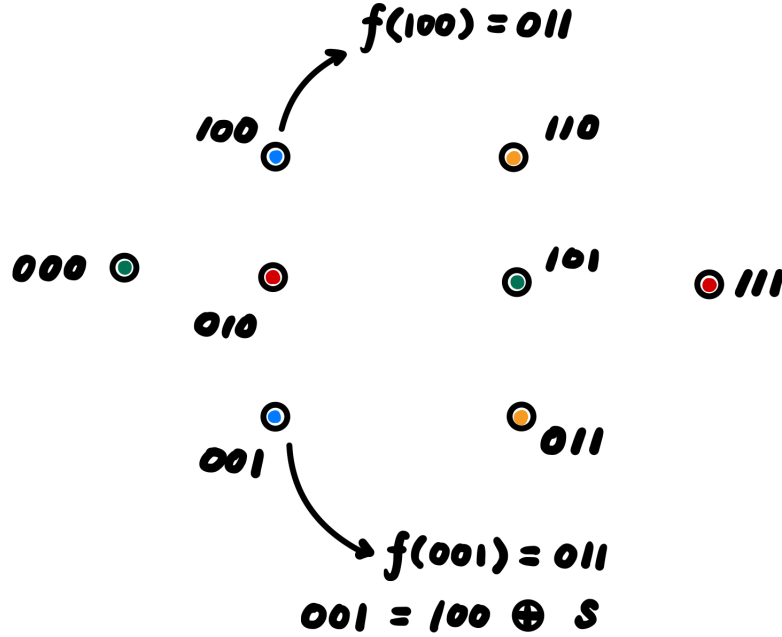
In some sense, the problem remains hard classically and easy quantumly, even when the oracle is instantiated as a circuit as far as we know: because Shor's algorithm is "in some sense" Simon's problem but with the oracle instantiated as a specific circuit.

The setup for the problem is the following. For $n \in \mathbb{N}$, let:

$$\mathcal{D}_0 := \{f : \{0, 1\}^n \rightarrow \{0, 1\}^n \mid f \text{ is a bijection}\}$$

$$\mathcal{D}_1 := \{f : \{0, 1\}^n \rightarrow \{0, 1\}^n \mid \exists! \mathbf{s} \in \{0, 1\}^n - \{0\}^n, \forall \mathbf{x} \in \{0, 1\}^n, f(\mathbf{x}) = f(\mathbf{x} \oplus \mathbf{s})\}$$

That is, in the case of $\mathcal{D}_1$, if $f(\mathbf{x}) = f(\mathbf{y})$, then $\mathbf{y} \in \{\mathbf{x}, \mathbf{x} \oplus \mathbf{s}\}$, where $\mathbf{s}$ is a unique non-zero bitstring. An example of this is:



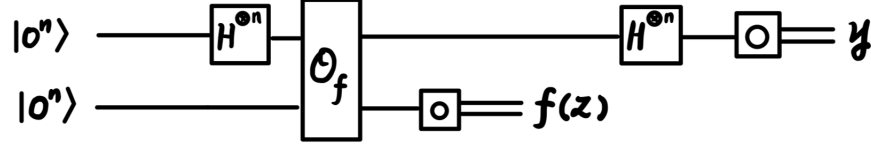
where $\mathbf{s} = 101$. We can view this function as a coloring with 2^{n-1} colors with each color covering exactly two inputs. Each color represent an output.

We want to decide which case f is in, given query access to f , *promised* one of the cases hold (but we do not know $\mathbf{s}$).

Classically, we need $2^{n-1} + 1$ queries to deterministically solve the problem. When employing a randomized algorithm, the time complexity is $\Theta(\sqrt{2^n})$, which is related to the "birthday paradox".

7.1 Quantum Strategy

In the quantum case, we only need $\mathcal{O}(n)$ queries and give the correct answer with high probability. The circuit is as follows:



We measure the last n qubits, and obtain $f(\mathbf{z})$. If $f \in \mathcal{D}_0$, the state of first n qubits collapses to $|\mathbf{z}\rangle$. If $f \in \mathcal{D}_1$, the state of first n qubits collapses to $\frac{1}{\sqrt{2}}(|\mathbf{z}\rangle + |\mathbf{z} \oplus \mathbf{s}\rangle)$. Then, after applying H^n , if $f \in \mathcal{D}_0$, then we have:

$$|\mathbf{z}\rangle \mapsto \frac{1}{\sqrt{2^n}} \sum_{\mathbf{y}} (-1)^{\mathbf{y} \cdot \mathbf{z}} |\mathbf{y}\rangle$$

if $f \in \mathcal{D}_1$, then:

$$\frac{1}{\sqrt{2}}(|\mathbf{z}\rangle + |\mathbf{z} \oplus \mathbf{s}\rangle) \mapsto \frac{1}{\sqrt{2^{n+1}}} \left(\sum_{\mathbf{y}} (-1)^{\mathbf{y} \cdot \mathbf{z}} (1 + (-1)^{\mathbf{y} \cdot \mathbf{s}}) |\mathbf{y}\rangle \right)$$

This means, if we measure the first n qubits, if f is in the first case, each $\mathbf{y}$ has uniform probability of being measured. If f is in the second case, then:

1. $\mathbf{y} \cdot \mathbf{s} \equiv 0 \pmod{2}$: $\Pr(\mathbf{y}) = \frac{1}{2^{n-1}}$.
2. $\mathbf{y} \cdot \mathbf{s} \equiv 1 \pmod{2}$: $\Pr(\mathbf{y}) = 0$

which is uniform over $\{\mathbf{y} \in \{0, 1\}^n \mid \mathbf{y} \cdot \mathbf{s} \equiv 0 \pmod{2}\}$.

Repeat the above process $K = \Theta(n) > n$ times to generate K $\mathbf{y}$'s. Collect these $\mathbf{y}$'s into a $K \times n$ matrix:

$$A = \begin{pmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \\ \vdots \\ \mathbf{y}_K \end{pmatrix}$$

and compute the rank of A over $\mathbb{F}_2$. If the rank is n , output $\mathcal{D}_0$, if the rank is $< n$, output $\mathcal{D}_1$.

Definition 7.1. For a matrix A in $\mathbb{F}_2$, its rank is the minimal non-negative d such that the set of rows of A can be expressed as

$$\{\beta_1 \mathbf{v}_1 \oplus \beta_2 \mathbf{v}_2 \oplus \cdots \beta_d \mathbf{v}_d \mid \beta_1, \beta_2, \dots, \beta_d \in \{0, 1\}\}$$

for some zero-one $\mathbf{v}_1, \dots, \mathbf{v}_d$.

For example, take $\mathbf{v}_1 = (1, 0, 1)$ and $\mathbf{v}_2 = (1, 1, 0)$, by iterating through $\beta_1, \beta_2 \in \{0, 1\}$, we can obtain 3 non-zero vectors.

This gives that the algorithm is always correct in $\mathcal{D}_1$.

However, for $f \in \mathcal{D}_0$, we have:

Lemma 7.1.

$$\Pr(\text{rank } A = n) \geq 1 - 2^{n-K}$$

Additionally, $\mathbf{s}$ can be computed from $\ker A$.

8 Shor's Algorithm

8.1 Modulo Operations and Relevant Mathematical Background

Definition 8.1. A prime number $p \in \mathbb{Z}^+$ is a number that has no divisors other than 1 and p .

Theorem 8.1. The *Fundamental Theorem of Arithmetic* states that for all positive integers x , x can be factored into a product of primes. Moreover, the factorization is unique (up to re-ordering).

Given a positive integer N , let $x, y \in \mathbb{Z}$, we write

$$x \bmod N$$

to mean the unique element $x' \in \{0, \dots, N-1\}$ such that N divides $(x - x')$.

We write

$$x \equiv y \pmod{N}$$

to mean that $x \bmod N = y \bmod N$.

Definition 8.2. Given $a, b \in \mathbb{Z}$, not both zero, the *greatest common divisor* of a and b is the largest $d \in \mathbb{Z}^+$ such that d divides a and d divides b . We write:

$$\gcd(a, b) = d$$

Definition 8.3. a, b are co-prime if $\gcd(a, b) = 1$.

Definition 8.4. Let $a, N \in \mathbb{Z}^+$, if $\gcd(a, N) = 1$, and $\exists r \in \mathbb{Z}^+$ where $a^r \equiv 1 \pmod{N}$, the least such r is called *the order of a modulo N* , denoted as

$$\text{ord}_N(a)$$

We only consider the case where $N = pq$, for p, q being distinct primes.

We describe the algorithm in three steps:

1. Choose a uniformly at random from the set $\{1, \dots, N-1\}$, compute $d := \gcd(a, N)$. If $d > 1$, output d . Otherwise, we know a and N are co-prime.
2. Compute $r := \text{ord}_N(a)$ *on a quantum computer*. If r is odd, output “do not know”, otherwise, continue to Step 3.
3. Compute $d' := \gcd(a^{\frac{r}{2}} - 1 \bmod N, N)$. If $d' > 1$, output d' . Otherwise, output “do not know” and the algorithm fails for this instance.

This algorithm will always output the correct answer if it does not output “do not know”.

Theorem 8.2. Suppose a is chosen u.a.r from $\{a' \in \{1, \dots, N-1\} \mid \gcd(a', N) = 1\}$, then

$$\Pr(2|r := \text{ord}_N(a) \wedge a^{\frac{r}{2}} \neq -1 \pmod{N}) \geq \frac{1}{2}$$

Theorem 8.3.

$$\Pr(\text{"do not know"}) \leq \frac{1}{2}$$

8.2 Quantum Fourier Transform

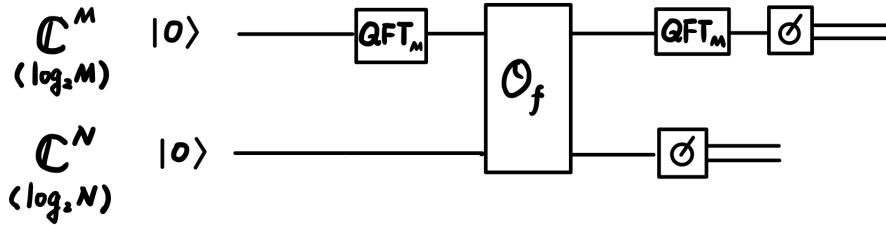
Definition 8.5. Let $M \in \mathbb{Z}^+$, the quantum fourier transform on $\mathbb{C}^M$ is the unitary defined by

$$\text{QFT}_M |j\rangle = \frac{1}{\sqrt{M}} \sum_{k=0}^{M-1} \omega_M^{k \cdot j} |k\rangle$$

for $j \in \{0, 1, \dots, M-1\}$ and

$$\omega_M = \exp\left(i \frac{2\pi}{M}\right)$$

The quantum circuit that computes $r := \text{ord}_N(a)$ is the following:



where we have:

$$f : \{0, 1, 2, \dots, M-1\} \rightarrow \{0, 1, \dots, N-1\}$$

$$x \mapsto a^x \pmod{N}$$

We take $M > N^2$ and assume $M = \lambda r$ for some $\lambda \in \mathbb{Z}^+$.

After we measure the second qubit and obtain some $f(k)$ for $k \in \{0, 1, \dots, r-1\}$, the first qubit collapses to:

$$\frac{1}{\sqrt{\lambda}} \sum_{l=0}^{\lambda-1} |k + lr\rangle$$

where $l \in \{0, 1, \dots, \lambda-1\}$.

After applying QFT on the first qubit, measurement on the first qubit, gives $n\lambda$ where n is chosen u.a.r from $\{0, 1, \dots, r-1\}$.

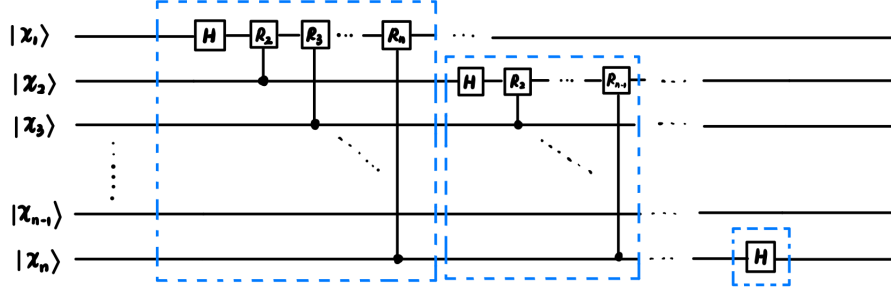
We extract r by the following:

1. Divide $n\lambda$ by M to get $f := \frac{n}{r}$.
2. Express f in its simplest terms and read off the denominator, call r' .
3. Verify if $a^{r'} \equiv 1 \pmod{N}$, if yes give r' , if not give “fail”.

There are two cases, if n is co-prime to r , then $r' = r$, which occurs with probability $\geq \gamma$; if n is not co-prime to r , r is a multiple of r' . Then doing $\mathcal{O}\left(\frac{1}{\gamma}\right)$ repeats of the algorithm ensures with almost certainty that we land in the first case and obtain r .

The complexity of each repeat is $\mathcal{O}(\log^3 N)$ and $\frac{1}{\gamma} \sim \log(\log N)$.

The QFT circuit is implemented in the following way:



where

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\frac{2\pi}{2^k}} \end{pmatrix}$$

This circuit is realized using $\mathcal{O}(n^2) \sim \mathcal{O}(\log^2 N)$ quantum gates.

9 Grover's Algorithm

Grover's algorithm can be used to speed up the SAT problem.

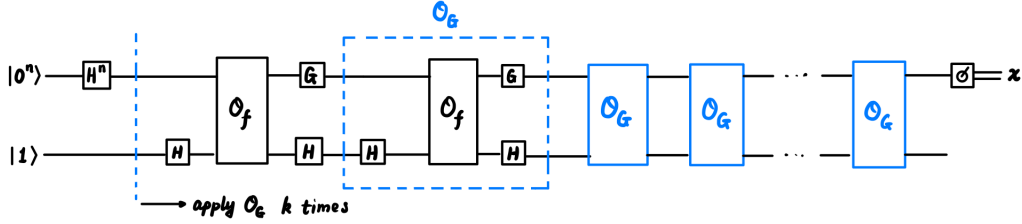
Let

$$f : \{0, 1\}^n \rightarrow \{0, 1\}$$

we restrict it to

$$\begin{cases} f(\mathbf{x}) = 0 & \forall \mathbf{x} \in \{0, 1\}^n \\ f(\mathbf{x}) = 0 & \exists! \mathbf{x}^* \in \{0, 1\}^n, f(\mathbf{x}^*) = 1 \end{cases}$$

The quantum circuit for Grover's algorithm is the following:



Recall the quantum phase oracle for f is

$$\mathcal{U}_f = H O_f H : |\mathbf{x}\rangle |b\rangle \mapsto (-1)^{b \cdot f(\mathbf{x})} |\mathbf{x}\rangle |b\rangle$$

We define V_f where (take $b = 1$ for $\mathcal{U}_f$)

$$V_f : |\mathbf{x}\rangle \mapsto (-1)^{f(\mathbf{x})} |\mathbf{x}\rangle$$

and define:

$$|\psi\rangle = \frac{1}{\sqrt{2^n}} \sum_{\mathbf{x}} |\mathbf{x}\rangle$$

$$G := \mathbb{I} - 2|\psi\rangle\langle\psi|$$

The state before measurment in this setup is essentially

$$(G V_f)^k H^n |0^n\rangle$$

In the first case, this becomes $(-1)^k |\psi\rangle$.

In the second case, $|\psi\rangle$ can be written as:

$$|\psi\rangle = \sqrt{\frac{N-1}{N}} |\psi_0\rangle + \sqrt{\frac{1}{N}} |\psi_1\rangle = \cos \theta |\psi_0\rangle + \sin \theta |\psi_1\rangle$$

where $N = 2^n$, and

$$\begin{aligned} |\psi_0\rangle &= \frac{1}{\sqrt{N-1}} \sum_{\mathbf{x} \neq \mathbf{x}^*} |\mathbf{x}\rangle \\ |\psi_1\rangle &= |\mathbf{x}^*\rangle \end{aligned}$$

We can calculate $-GV_f$ with respect to $|\psi_0\rangle$ and $|\psi_1\rangle$ to be:

$$\begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix}$$

which is rotating 2θ counterclockwise.

In the second case, $|\psi\rangle$ starts with a θ between it and the $|\psi_0\rangle$ axis, and after applying $-GV_f$ k times, we want it to be close to $|\psi_1\rangle$. This means:

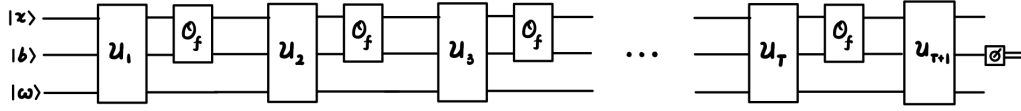
$$(2k+1)\theta = \frac{\pi}{2} \rightarrow k \approx \frac{\pi}{4}\sqrt{N}$$

(Soufflé problem: fixed point amplification)

9.1 Limitations of Quantum Algorithm in Query Models

We demonstrate that if a quantum computer can only query f , it must use $\Omega(\sqrt{2^n})$ queries to decide if f maps $\mathbf{x}$ to 1.

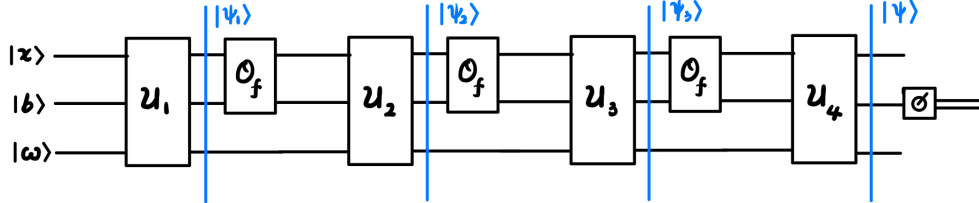
We consider a general model of any quantum algorithm:



Here, T represents the number of queries to f .

We consider the circuit for different choices of f , denote the circuit where f maps everything to 0 to be “0”, f maps $\mathbf{x}_i$ to 1 and the rest to 0 to be “ $\mathbf{x}_i$ ”.

The limitation is that if T is small, the algorithm can “look at” at most T^2 values of $f(\mathbf{x})$, and if $T^2 < N$, there will exist $\mathbf{x}^* \neq \mathbf{x}_i$ such that the output of circuit “0” is indistinguishable from “ $\mathbf{x}^*$ ”.



Let $t \in \{1, \dots, T\}$ and $|\psi_t\rangle$ be the state for t th query, we write:

$$|\psi_t\rangle = \sum_{\mathbf{x}, b, \omega} \alpha_{\mathbf{x}, b, \omega}^t |\mathbf{x}, b, \omega\rangle$$

where t is just a **superscript**.

Let

$$Q_{\mathbf{x}}^t = \sum_{b, \omega} |\alpha_{\mathbf{x}, b, \omega}^t|^2$$

$$Q_{\mathbf{x}} = \sum_{t=1}^T Q_{\mathbf{x}}^t$$

Then, by expanding the terms:

$$\sum_{\mathbf{x}} Q_{\mathbf{x}} = T$$

there must exist $\mathbf{x}^*$ such that $Q_{\mathbf{x}^*} \leq \frac{T}{N}$. This means, if $T < \sqrt{N}$, the quantum circuit cannot distinguish circuit “0” from circuit “ $\mathbf{x}^*$ ”.

We show this by considering a function, $g : \{0, 1\}^n \rightarrow \{0, 1\}$ such that $g(\mathbf{x}^*) = 1$ and $g(\mathbf{x}) = 0$ for $\mathbf{x} \neq \mathbf{x}^*$.

Theorem 9.1. Let $|\gamma\rangle$ be the state of circuit “ $\mathbf{x}$ ” just before measurement, Then

$$\| |\psi\rangle - |\gamma\rangle \| \leq 2 \sum_{t=1}^T \sqrt{Q_{\mathbf{x}^*}^t}$$

The idea behind this is that we first replace the last f from circuit “0” with g and bound the difference. Then replace the last two f from circuit “0” with g and bound the difference. Eventually, we replace all f with g and bound the difference.

Theorem 9.2. At time step t , we replace the last $T - t + 1$ f with g , and we can show that at each replacement, the difference between states just before measurement is bound by $4Q_{\mathbf{x}}^t$

Then:

$$\begin{aligned} \| |\psi\rangle - |\gamma\rangle \| &\leq 2 \sum_{t=1}^T \sqrt{Q_{\mathbf{x}}^t} \\ &= 2 \sum_{t=1}^T 1 \cdot \sqrt{Q_{\mathbf{x}}^t} \\ &\leq 2\sqrt{T} \sqrt{\sum_{t=1}^T Q_{\mathbf{x}}^t} \\ &= \frac{2T}{\sqrt{N}} \end{aligned}$$

which is small unless $T \geq \Omega(\sqrt{N})$.

10 Quantum Error Detection and Correction - 量子错误探测与改正

10.1 Classical Error Correction

The fundamental idea of error detection and correction is to *add redundancy to data to project against errors*.

The simplest classical code is “repetition code”, where we code 0 as 000 and 1 as 111. We can detect and correct any single bit flip. The rate of this method is $\frac{1}{3}$.

Another classical method is the Hamming code, where we code 4 bits into 7 bits, with the rate of $\frac{4}{7}$. This is achieved by

$$p_1 = d_1 \oplus d_2 \oplus d_3$$

for one of the error detection code.

10.2 Quantum Error Detection and Correction

A naïve generalization of the repetition code does not work in quantum, the operation:

$$|\psi\rangle \mapsto |\psi\rangle |\psi\rangle |\psi\rangle$$

is not possible to due *no-cloning principle*.

Definition 10.1. A d -dimensional *observable* $\mathcal{O}$ is a $d \times d$ *Hermitian* matrix where $\mathcal{O}^\dagger = \mathcal{O}$.

An n -qubit observable is a 2^n -dimensional observable.

Definition 10.2. Let $\mathcal{O}$ be an n -qubit observable, $|\psi\rangle$ is an n -qubit state, suppose $\mathcal{O}|\psi\rangle = \lambda|\psi\rangle$ for some $\lambda \in \mathbb{R}$, then *measuring* $\mathcal{O}$ on $|\psi\rangle$ refers to a process that returns λ and the state remains $|\psi\rangle$.

Definition 10.3. An n -qubit *stabilizer* (Pauli operators) is a $2^n \times 2^n$ matrix of form $P_1 \otimes P_2 \otimes \cdots \otimes P_n$, where $P_i \in \{\mathbb{I}, X, Y, Z\}$

When we measure a stabilizer on an eigenstate, we will obtain outcomes in $\{+1, -1\}$.

Definition 10.4. If A, B are complex matrices of same dimensions, we say A, B *commute* if $AB = BA$ and *anti-commute* if $AB = -BA$.

This gives that any two stabilizers commute or anti-commute.

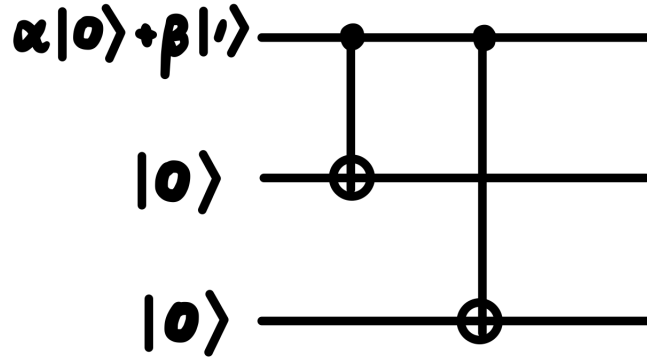
A set of k stabilizers can be *simultaneously* measured if $\mathcal{S}_i$ commutes with $\mathcal{S}_j$. This is a “physical interpretation” of the mathematical fact that a set of commuting diagonalizable matrices can be simultaneously diagonalized.

Lemma 10.1. Let $|\psi\rangle$ be an n -qubit state and P an n -qubit stabilizer, and $\mathcal{U}$ an n -qubit unitary, if $P|\psi\rangle = |\psi\rangle$ (P stabilizes $|\psi\rangle$), and $P\mathcal{U} = \alpha\mathcal{U}P$, for $\alpha \in \{+1, -1\}$, then measuring P on $\mathcal{U}|\psi\rangle$ gives outcome α .

10.2.1 Bit-flip Error

Analogous to the repetition code, we encode a state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ to be $|\psi'\rangle = \alpha|000\rangle + \beta|111\rangle$, where $|000\rangle := |0_L\rangle$ and $|111\rangle = |1_L\rangle$ (“L”: logical encoding).

This is achieved through the circuit:



This protects against any single-qubit flip. We can detect and correct the errors by measuring:

$$Z_1 Z_2 = Z \otimes Z \otimes \mathbb{I}$$

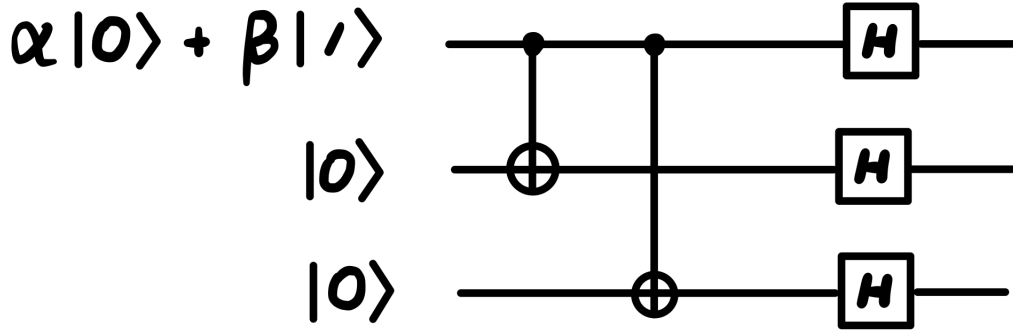
$$Z_2 Z_3 = \mathbb{I} \otimes Z \otimes Z$$

If we measure to $(+, +)$, there is no error. If we measure to $(-, +)$, the first qubit is flipped. If we measure to $(-, -)$, the second qubit is flipped. If we measure to $(+, -)$, the third qubit is flipped. We call this the “error syndrome table”.

However, this cannot detect phase flips.

10.2.2 Phase-flip Error

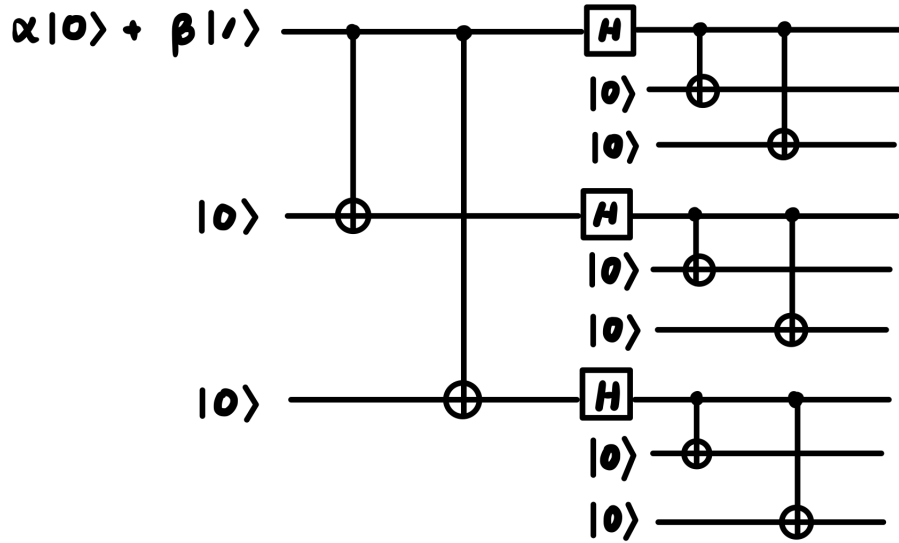
We encode a state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ to be $|\psi'\rangle = \alpha|+++ \rangle + \beta|--- \rangle$, where $|+++ \rangle := |0_L\rangle$ and $|--- \rangle = |1_L\rangle$ through the circuit:



We then measure (X_1X_2, X_2X_3) . Analogous to previous section, this can detect any single-qubit phase flip, not any single-qubit bit flip.

10.2.3 Shor's Code

One way to detect both bit flips and phase flips on a single-qubit is to use 9-qubit Shor's code, where it is achieved by the circuit:



And the logical encoding is that

$$|0_L\rangle := \frac{1}{2\sqrt{2}}(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)$$

$$|1_L\rangle := \frac{1}{2\sqrt{2}}(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)$$

The corresponding stabilizers are:

$$(Z_1Z_2, Z_2Z_3), (Z_4Z_5, Z_5Z_6)(Z_7Z_8, Z_8Z_9)$$

and

$$(X_1X_2X_3X_4X_5X_6, X_4X_5X_6X_7X_8X_9)$$

Even though Shor's code's error syndrome table has non-distinct columns, we can still correct any single qubit error.

For example, the columns Z_1, Z_2, Z_3 (error gates) will be the same, if we observe the common syndrome of Z_1, Z_2, Z_3 , we can still correct by multiplying the state by Z_1 .

1. Error was Z_1 : $Z_1(Z_1|\psi\rangle) = |\psi\rangle$.
2. Error was Z_2 : $Z_1(Z_2|\psi\rangle) = (Z_1Z_2)|\psi\rangle = |\psi\rangle$ as Z_1Z_2 is a stabilizer.
3. Error was Z_3 : $Z_1(Z_3|\psi\rangle) = (Z_1Z_2)(Z_2Z_3)|\psi\rangle = |\psi\rangle$

We can identify the block on which the error occurred:

1. if the Z rows have $-$ signs, 1 – 3 are the first block, 4 – 6 are the second block, and 7 – 9 are the third block, with X, Y errors.
2. if the X rows take value $(-, +)$, it's first block; if $(-, -)$, it's second block; if $(+, -)$, it's third block.

If a QEC can correct any single-qubit X, Y, Z errors, then it can correct “arbitrary” single-qubit errors.

10.3 How to Measure Stabilizers

We can measure say $I \otimes X \otimes Y \otimes Z$ on a 4-qubit state $|\psi\rangle$ in the following way:

